A) What is multitasking in Java? How is it achieved using threads?

Multitasking:
Multitasking means executing **multiple tasks at the same time**.

In Java:
It is mainly achieved using **multithreading**, where multiple threads run concurrently.

How it is achieved:

- By using **Thread class** (extends Thread)
- By using **Runnable interface** (implements Runnable)
- Each thread executes independently

Example (simple):

```
class MyThread extends Thread {
  public void run() {
    System.out.println("Thread is running");
  }

  public static void main(String args[]) {
    MyThread t1 = new MyThread();
    t1.start();
  }
```

B) Difference between AWT and Swing

Feature	AWT	Swing
Platform	Platform dependent	Platform independent
Components	Heavyweight	Lightweight
Look	Native OS look	Custom look
Package	java.awt	javax.swing
Flexibility	Less flexible	More flexible

A) Thread Synchronization in Java

Definition:
Thread synchronization is used to control multiple threads accessing the same resource to avoid inconsistency.

Key Points:

- Prevents data corruption

- Uses synchronized keyword
- Only one thread executes at a time

Program:

```
class Table {
  synchronized void print(int n) {
    for(int i=1;i<=5;i++) {
      System.out.println(n*i);
      try { Thread.sleep(400); } catch(Exception e){}
    }
  }
}

class T1 extends Thread {
  Table t;
  T1(Table t){ this.t=t; }
  public void run(){ t.print(5); }
}

class T2 extends Thread {
  Table t;
  T2(Table t){ this.t=t; }
  public void run(){ t.print(10); }
}

class Test {
  public static void main(String args[]) {
    Table obj = new Table();
    new T1(obj).start();
    new T2(obj).start();
  }
}
```

B) JLabel and JButton + Swing GUI

JLabel: Used to display text
JButton: Used to perform action on click

Program:

```
import javax.swing.*;
import java.awt.event.*;

class Demo {
  public static void main(String args[]) {
    JFrame f = new JFrame("Swing Example");
```

```
    JLabel l = new JLabel("Press Button");
    l.setBounds(100,50,150,30);

    JButton b = new JButton("Click");
    b.setBounds(100,100,100,30);

    b.addActionListener(new ActionListener() {
      public void actionPerformed(ActionEvent e) {
        l.setText("Button Clicked");
      }
    });

    f.add(l);
    f.add(b);

    f.setSize(300,300);
    f.setLayout(null);
    f.setVisible(true);
  }
}
```

C) JDBC Drivers + Steps

Definition:
JDBC is used to connect Java application with database.

Types of Drivers:

1. Type 1 – Bridge Driver
2. Type 2 – Native Driver
3. Type 3 – Network Driver
4. Type 4 – Thin Driver (most used)

Steps to connect JDBC:

1. Load driver
2. Create connection
3. Create statement
4. Execute query
5. Close connection

Program:

```
import java.sql.*;

class JDBCEx {
```

```
  public static void main(String args[]) {
    try {
      Class.forName("com.mysql.cj.jdbc.Driver");

      Connection con = DriverManager.getConnection(
      "jdbc:mysql://localhost:3306/test","root","1234");

      Statement stmt = con.createStatement();
      ResultSet rs = stmt.executeQuery("select * from student");

      while(rs.next()) {
        System.out.println(rs.getInt(1)+" "+rs.getString(2));
      }

      con.close();
    } catch(Exception e) {
      System.out.println(e);
    }
  }
}
```

D) Exception Propagation

Definition:
When an exception occurs, it moves from one method to another until it is handled.

Program:

```
class Test {
  void m() {
    int data = 10/0;
  }

  void n() {
    m();
  }

  void p() {
    try {
      n();
    } catch(Exception e) {
      System.out.println("Exception handled");
    }
  }

  public static void main(String args[]) {
    Test obj = new Test();
    obj.p();
```

```
  }
}
```

E) Mouse Events in Java

Definition:
Mouse events handle actions like click, move, press.

Methods:

- mouseClicked()
- mousePressed()
- mouseReleased()
- mouseEntered()
- mouseExited()

Program:

```
import javax.swing.*;
import java.awt.event.*;

class MouseDemo extends JFrame implements MouseListener {

  JLabel l;

  MouseDemo() {
    l = new JLabel();
    l.setBounds(100,100,200,30);
    add(l);

    addMouseListener(this);

    setSize(300,300);
    setLayout(null);
    setVisible(true);
  }
   public void mouseClicked(MouseEvent e) {
    l.setText("Mouse Clicked");
  } public void mouseEntered(MouseEvent e) {}
  public void mouseExited(MouseEvent e) {}
  public void mousePressed(MouseEvent e) {}
  public void mouseReleased(MouseEvent e) {}

  public static void main(String args[]) {
    new MouseDemo();
  }
}
```

Q. what is multithreading ? explain the life cycle of a thread with detailed diagram write a pgm to create a multithreaded application using both thread and runnable classes and explain flow?

Multithreading in Java

Definition:
Multithreading is a process of executing **multiple threads simultaneously** within a single program to improve performance and CPU utilization.

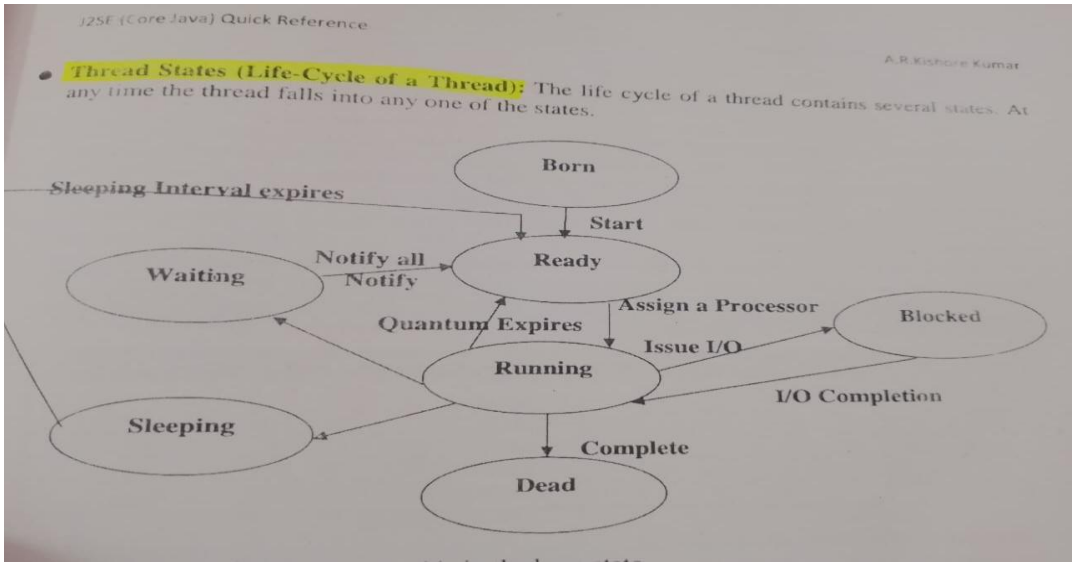
Key Points:

- Thread = lightweight process
- Threads share memory
- Used for multitasking (e.g., downloading + playing music)

✅ **Life Cycle of a Thread (with Diagram)**

States of Thread:

1. **New** – Thread is created
2. **Runnable** – Ready to run
3. **Running** – Currently executing
4. **Blocked/Waiting** – Waiting for resource
5. **Terminated** – Execution finished



Explanation of Flow:

- start() → moves thread from **New → Runnable**
- Scheduler selects → **Runnable → Running**
- sleep() / wait() → goes to **Blocked/Waiting**

- After completion → **Terminated**

✅ **Program: Multithreading using Thread & Runnable**

Using Thread Class

```
class MyThread extends Thread {
  public void run() {
    for(int i=1;i<=5;i++) {
      System.out.println("Thread Class: " + i);
      try { Thread.sleep(500); } catch(Exception e){}
    }
  }
}
```

Using Runnable Interface

```
class MyRunnable implements Runnable {
  public void run() {
    for(int i=1;i<=5;i++) {
      System.out.println("Runnable: " + i);
      try { Thread.sleep(500); } catch(Exception e){}
    }
  }
}
```

Main Class

```
class Test {
  public static void main(String args[]) {

    // Thread class object
    MyThread t1 = new MyThread();

    // Runnable object
    MyRunnable r = new MyRunnable();
    Thread t2 = new Thread(r);

    // Start threads
    t1.start();
    t2.start();
  }
}
```

✅ **Flow of Execution (Very Important for Exam)**

1. Program starts → main() method runs
2. Two threads are created:
    - t1 using Thread class
    - t2 using Runnable interface
3. start() method is called:
    - Moves threads to **Runnable state**
4. CPU scheduler executes both threads:
    - Output runs **simultaneously (interleaving)**
5. Each thread executes run() method
6. After loop completes → threads go to **Terminated state**

Q. what are layout managers in java? explain flowlayout,gridlayout & borderlayout .write a pgm to create a GUI with different layout managers.

✅ **Layout Managers in Java**

Definition:
Layout Managers are used to **arrange GUI components automatically** inside a container (like Frame or JFrame).

Why needed:

- Avoid manual positioning (setBounds)
- Adjust components dynamically
- Make GUI responsive

✅ **Types of Layout Managers**

---

1) FlowLayout

Definition:
Arranges components **left to right**, like a flow of text.

Features:

- Default layout for Panel
- Moves to next line if space is full
- Supports alignment (LEFT, CENTER, RIGHT)

Example:

setLayout(new FlowLayout());

---

2) GridLayout

Definition:
Arranges components in a **grid of rows and columns**.

Features:

- All components are equal size
- Fills grid completely
- Order: left to right, top to bottom

Example:

setLayout(new GridLayout(2, 2));

---

3) BorderLayout

Definition:
Divides container into **5 regions**:

- NORTH
- SOUTH
- EAST
- WEST
- CENTER

Features:

- Default layout for Frame
- CENTER takes remaining space

Example:

setLayout(new BorderLayout());

📊 **Summary Table**

Layout	Arrangement	Key Feature
FlowLayout	Left → Right	Simple flow
GridLayout	Rows × Columns	Equal size
BorderLayout	5 regions	Flexible

✅ **Program: GUI with Different Layout Managers**

```
import javax.swing.*;
import java.awt.*;

class LayoutDemo {

   public static void main(String args[]) {

      // Frame
      JFrame f = new JFrame("Layout Manager Demo");
      f.setSize(400,400);

      // ---------- FlowLayout ----------
      JPanel p1 = new JPanel();
      p1.setLayout(new FlowLayout());
      p1.add(new JButton("F1"));
      p1.add(new JButton("F2"));
      p1.add(new JButton("F3"));

      // ---------- GridLayout ----------
      JPanel p2 = new JPanel();
      p2.setLayout(new GridLayout(2,2));
      p2.add(new JButton("G1"));
      p2.add(new JButton("G2"));
      p2.add(new JButton("G3"));
      p2.add(new JButton("G4"));

      // ---------- BorderLayout ----------
      JPanel p3 = new JPanel();
      p3.setLayout(new BorderLayout());
      p3.add(new JButton("North"), BorderLayout.NORTH);
      p3.add(new JButton("South"), BorderLayout.SOUTH);
      p3.add(new JButton("East"), BorderLayout.EAST);
      p3.add(new JButton("West"), BorderLayout.WEST);
      p3.add(new JButton("Center"), BorderLayout.CENTER);

      // Add panels to frame
      f.setLayout(new GridLayout(3,1));
      f.add(p1);
      f.add(p2);
      f.add(p3);

      f.setVisible(true);
   }
}
```

Q. what is JDBC ? explain its architecture , write a program to insert and retrive records from MYSQL table using JDBC

✅ What is JDBC?

JDBC (Java Database Connectivity) is an API used to **connect Java applications with databases** like MySQL, Oracle, etc.

Key Points:

- Provides methods to execute SQL queries
- Located in java.sql package
- Supports CRUD operations (Create, Read, Update, Delete)

✅ JDBC Architecture

JDBC follows a **client-server architecture**.

📊 Architecture Diagram

Java Application
|
v
JDBC API
|
v
JDBC Driver Manager
|
v
JDBC Driver
|
v
Database (MySQL)

Components Explanation:

1. **Java Application**
    - User program that uses JDBC
2. **JDBC API**
    - Interfaces like Connection, Statement, ResultSet
3. **DriverManager**
    - Manages database drivers and connections
4. **JDBC Driver**

- Converts Java calls into database-specific commands

5. **Database**
   - Stores actual data (MySQL)

✅ **Steps to Connect JDBC**

1. Load Driver
2. Establish Connection
3. Create Statement
4. Execute Query
5. Close Connection

✅ **Program: Insert and Retrieve Data from MySQL**

🔹 **Database Table Example**

```
CREATE TABLE student (
  id INT,
  name VARCHAR(50)
);
```

🔹 **Java Program**

```
import java.sql.*;

class JDBCExample {
  public static void main(String args[]) {
    try {
      // 1. Load Driver
      Class.forName("com.mysql.cj.jdbc.Driver");

      // 2. Create Connection
      Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/test","root","1234");

      // 3. Create Statement
      Statement stmt = con.createStatement();

      // 4. Insert Record
      stmt.executeUpdate("insert into student values(1,'Chetan')");
      stmt.executeUpdate("insert into student values(2,'Rahul')");
```

```
        // 5. Retrieve Records
        ResultSet rs = stmt.executeQuery("select * from student");

        while(rs.next()) {
          System.out.println(rs.getInt(1) + " " + rs.getString(2));
        }

        // 6. Close Connection
        con.close();

      } catch(Exception e) {
        System.out.println(e);
      }
    }
}
```

Q.what are streams in java ? Write a pgm to copy content one file to another

✅ **Streams in Java**

Definition:
A **stream** in Java is a sequence of data used to **read (input) or write (output)** data between a program and a source/destination (like file, keyboard, network).

Types of Streams

1) Input Stream

- Used to **read data**
- Example: FileInputStream

2) Output Stream

- Used to **write data**
- Example: FileOutputStream

Classification

Type	Description
Byte Streams	Handle binary data (images, files)
Character Streams	Handle text data

Examples

- FileInputStream
- FileOutputStream
- FileReader
- FileWriter

✅ **Program: Copy Content from One File to Another**

Using FileInputStream & FileOutputStream (Byte Stream)

```
import java.io.*;

class FileCopy {
  public static void main(String args[]) {
    try {
      // Input file
      FileInputStream fin = new FileInputStream("input.txt");

      // Output file
      FileOutputStream fout = new FileOutputStream("output.txt");

      int i;

      // Read and write byte by byte
      while((i = fin.read()) != -1) {
        fout.write(i);
      }

      fin.close();
      fout.close();

      System.out.println("File copied successfully");

    } catch(Exception e) {
      System.out.println(e);
    }
  }
}
```

Q.what is the difference between throw & throws ? write a pgm to demonstrate both with user defined exception.

✅ **Difference between throw and throws in Java**

Feature	throw	throws
Purpose	Used to **explicitly throw an exception**	Used to **declare exceptions**
Usage	Inside method body	In method declaration
Number of exceptions	Throws **one exception at a time**	Can declare **multiple exceptions**
Keyword type	Statement	Clause
Example	throw new Exception();	void m() throws Exception

✅ User Defined Exception

A **user-defined exception** is created by extending the Exception class.

✅ Program: Demonstrate throw and throws

```
class MyException extends Exception {
  MyException(String msg) {
    super(msg);
  }
}

class Test {

  // Using throws
  static void checkAge(int age) throws MyException {
    if(age < 18) {
      // Using throw
      throw new MyException("Not eligible to vote");
    } else {
      System.out.println("Eligible to vote");
    }
  }

  public static void main(String args[]) {
    try {
      checkAge(16);
    } catch(MyException e) {
      System.out.println("Exception: " + e.getMessage());
    }
  }
}
```

✅ **Output**

Exception: Not eligible to vote

Q.write a pgm in java creating frame and closing frame?

Using AWT (Frame + WindowListener)

```
import java.awt.*;
import java.awt.event.*;

class MyFrame extends Frame implements WindowListener {

  MyFrame() {
    setTitle("My Frame");
    setSize(300, 300);
    setVisible(true);

    // Register window listener
    addWindowListener(this);
  }

  // Method to close frame
  public void windowClosing(WindowEvent e) {
    System.out.println("Closing window...");
    dispose(); // closes the frame
  }

  // Other methods (must override)
  public void windowOpened(WindowEvent e) {}
  public void windowClosed(WindowEvent e) {}
  public void windowIconified(WindowEvent e) {}
  public void windowDeiconified(WindowEvent e) {}
  public void windowActivated(WindowEvent e) {}
  public void windowDeactivated(WindowEvent e) {}

  public static void main(String args[]) {
    new MyFrame();
  }
}
```

Alternative (Shortcut using WindowAdapter)

```
import java.awt.*;
import java.awt.event.*;

class Test {
  public static void main(String args[]) {

    Frame f = new Frame("Simple Frame");
```

```
    f.setSize(300,300);
    f.setVisible(true);

    f.addWindowListener(new WindowAdapter() {
      public void windowClosing(WindowEvent e) {
        f.dispose();
      }
    });
  }
}
```

Q. write a pgm in java displaying image in frame

✅ **Program: Display Image in Frame (Using AWT)**

```
import java.awt.*;
import java.awt.event.*;

class ImageFrame extends Frame {

  Image img;

  ImageFrame() {
    // Load image
    img = Toolkit.getDefaultToolkit().getImage("image.jpg");

    setTitle("Display Image");
    setSize(400, 400);
    setVisible(true);

    // Close window
    addWindowListener(new WindowAdapter() {
      public void windowClosing(WindowEvent e) {
        dispose();
      }
    });
  }

  // Draw image
  public void paint(Graphics g) {
    g.drawImage(img, 50, 50, 300, 300, this);
  }

  public static void main(String args[]) {
    new ImageFrame();
  }
}
```

Q. write a java pgm displaying text in the frame

✅ **Program: Display Text in Frame (Using AWT)**

```
import java.awt.*;
import java.awt.event.*;

class TextFrame extends Frame {

  TextFrame() {
    setTitle("Display Text");
    setSize(400, 300);
    setVisible(true);

    // Close window
    addWindowListener(new WindowAdapter() {
      public void windowClosing(WindowEvent e) {
        dispose();
      }
    });
  }

  // Display text
  public void paint(Graphics g) {
    g.drawString("Hello Chetan! Welcome to Java", 100, 150);
  }

  public static void main(String args[]) {
    new TextFrame();
  }
}
```